
CHAPTER 3

GENERATIVE AND DISCRIMINATIVE CLASSIFIERS: NAIVE BAYES AND LOGISTIC REGRESSION

Machine Learning

Copyright ©2017. Tom M. Mitchell. All rights reserved.

DRAFT OF October 1, 2020

PLEASE DO NOT DISTRIBUTE WITHOUT AUTHOR'S PERMISSION

This is a rough draft chapter intended for inclusion in the upcoming second edition of the textbook *Machine Learning*, T.M. Mitchell, McGraw Hill. You are welcome to use this for educational purposes, but do not duplicate or repost it on the internet. To distribute this to others, simply point them to <http://www.cs.cmu.edu/~tom/mlbook/NBayesLogReg.pdf>. For online copies of this and other materials related to this book, visit the web site www.cs.cmu.edu/~tom/mlbook.html.

Please send suggestions for improvements, or suggested exercises, to Tom.Mitchell@cmu.edu.

1 Learning Classifiers based on Bayes Rule

Here we consider the relationship between supervised learning, or function approximation problems, and Bayesian reasoning. We begin by considering how to design learning algorithms based on Bayes rule.

Consider a supervised learning problem in which we wish to approximate an unknown target function $f : X \rightarrow Y$, or equivalently $P(Y|X)$. To begin, we will assume Y is a boolean-valued random variable, and X is a vector containing n boolean attributes. In other words, $X = \langle X_1, X_2, \dots, X_n \rangle$, where X_i is the boolean random variable denoting the i th attribute of X .

Applying Bayes rule, we see that $P(Y = y_i|X)$ can be represented as

$$P(Y = y_i | X = x_k) = \frac{P(X = x_k | Y = y_i)P(Y = y_i)}{\sum_j P(X = x_k | Y = y_j)P(Y = y_j)}$$

where y_m denotes the m th possible value for Y , x_k denotes the k th possible vector value for X , and where the summation in the denominator is over all legal values of the random variable Y .

One way to learn $P(Y|X)$ is to use the training data to estimate $P(X|Y)$ and $P(Y)$. We can then use these estimates, together with Bayes rule above, to determine $P(Y|X = x_k)$ for any new instance x_k .

A NOTE ON NOTATION: We will consistently use upper case symbols (e.g., X) to refer to random variables, including both vector and non-vector variables. If X is a vector, then we use subscripts (e.g., X_i to refer to each random variable, or feature, in X). We use lower case symbols to refer to *values* of random variables (e.g., $X_i = x_{ij}$ may refer to random variable X_i taking on its j th possible value). We will sometimes abbreviate by omitting variable names, for example abbreviating $P(X_i = x_{ij} | Y = y_k)$ to $P(x_{ij} | y_k)$. We will write $E[X]$ to refer to the expected value of X . We use superscripts to index training examples (e.g., X_i^j refers to the value of the random variable X_i in the j th training example.). We use $\delta(x)$ to denote an “indicator” function whose value is 1 if its logical argument x is true, and whose value is 0 otherwise. We use the $\#D\{x\}$ operator to denote the number of elements in the set D that satisfy property x . We use a “hat” to indicate estimates; for example, $\hat{\theta}$ indicates an estimated value of θ .

1.1 Unbiased Learning of Bayes Classifiers is Impractical

If we are going to train a Bayes classifier by estimating $P(X|Y)$ and $P(Y)$, then it is reasonable to ask how much training data will be required to obtain reliable estimates of these distributions. Let us assume training examples are generated by drawing instances at random from an unknown underlying distribution $P(X)$, then allowing a teacher to label this example with its Y value.

A hundred independently drawn training examples will usually suffice to obtain a maximum likelihood estimate of $P(Y)$ that is within a few percent of its correct value¹ when Y is a boolean variable. However, accurately estimating $P(X|Y)$ typically requires many more examples. To see why, consider the number of parameters we must estimate when Y is boolean and X is a vector of n boolean attributes. In this case, we need to estimate a set of parameters

$$\theta_{ij} \equiv P(X = x_i | Y = y_j)$$

where the index i takes on 2^n possible values (one for each of the possible vector values of X), and j takes on 2 possible values. Therefore, we will need to estimate

¹Why? See Chapter 5 of edition 1 of *Machine Learning*.

approximately 2^{n+1} parameters. To calculate the exact number of required parameters, note for any fixed j , the sum over i of θ_{ij} must be one. Therefore, for any particular value y_j , and the 2^n possible values of x_i , we need compute only $2^n - 1$ independent parameters. Given the two possible values for Y , we must estimate a total of $2(2^n - 1)$ such θ_{ij} parameters. Unfortunately, this corresponds to two distinct parameters for *each* of the distinct instances in the instance space for X . Worse yet, to obtain reliable estimates of each of these parameters, we will need to observe each of these distinct instances multiple times! This is clearly unrealistic in most practical learning domains. For example, if X is a vector containing 30 boolean features, then we will need to estimate more than 3 billion parameters.

2 Naive Bayes Algorithm

Given the intractable sample complexity for learning Bayesian classifiers, we must look for ways to reduce this complexity. The Naive Bayes classifier does this by making a conditional independence assumption that dramatically reduces the number of parameters to be estimated when modeling $P(X|Y)$, from our original $2(2^n - 1)$ to just $2n$.

2.1 Conditional Independence

Definition: Given three sets of random variables X, Y and Z , we say X is **conditionally independent** of Y given Z , if and only if the probability distribution governing X is independent of the value of Y given Z ; that is

$$(\forall i, j, k) P(X = x_i | Y = y_j, Z = z_k) = P(X = x_i | Z = z_k)$$

As an example, consider three boolean random variables to describe the current weather: *Rain*, *Thunder* and *Lightning*. We might reasonably assert that *Thunder* is independent of *Rain* given *Lightning*. Because we know *Lightning* causes *Thunder*, once we know whether or not there is *Lightning*, no additional information about *Thunder* is provided by the value of *Rain*. Of course there is a clear dependence of *Thunder* on *Rain* in general, but there is no *conditional* dependence once we know the value of *Lightning*. Although X, Y and Z are each single random variables in this example, more generally the definition applies to sets of random variables. For example, we might assert that variables $\{A, B\}$ are conditionally independent of $\{C, D\}$ given variables $\{E, F\}$.

2.2 Derivation of Naive Bayes Algorithm

The Naive Bayes algorithm is a classification algorithm based on Bayes rule and a set of conditional independence assumptions. Given the goal of learning $P(Y|X)$ where $X = \langle X_1 \dots, X_n \rangle$, the Naive Bayes algorithm makes the assumption that

each X_i is conditionally independent of each of the other X_k s given Y , and also independent of each subset of the other X_k 's given Y .

The value of this assumption is that it dramatically simplifies the representation of $P(X|Y)$, and the problem of estimating it from the training data. Consider, for example, the case where $X = \langle X_1, X_2 \rangle$. In this case

$$\begin{aligned} P(X|Y) &= P(X_1, X_2|Y) \\ &= P(X_1|X_2, Y)P(X_2|Y) \\ &= P(X_1|Y)P(X_2|Y) \end{aligned}$$

Where the second line follows from a general property of probabilities, and the third line follows directly from our above definition of conditional independence. More generally, when X contains n attributes which satisfy the conditional independence assumption, we have

$$P(X_1 \dots X_n|Y) = \prod_{i=1}^n P(X_i|Y) \quad (1)$$

Notice that when Y and the X_i are boolean variables, we need only $2n$ parameters to define $P(X_i = x_{ik}|Y = y_j)$ for the necessary i, j, k . This is a dramatic reduction compared to the $2(2^n - 1)$ parameters needed to characterize $P(X|Y)$ if we make no conditional independence assumption.

Let us now derive the Naive Bayes algorithm, assuming in general that Y is any discrete-valued variable, and the attributes $X_1 \dots X_n$ are any discrete or real-valued attributes. Our goal is to train a classifier that will output the probability distribution over possible values of Y , for each new instance X that we ask it to classify. The expression for the probability that Y will take on its k th possible value, according to Bayes rule, is

$$P(Y = y_k|X_1 \dots X_n) = \frac{P(Y = y_k)P(X_1 \dots X_n|Y = y_k)}{\sum_j P(Y = y_j)P(X_1 \dots X_n|Y = y_j)}$$

where the sum is taken over all possible values y_j of Y . Now, assuming the X_i are conditionally independent given Y , we can use equation (1) to rewrite this as

$$P(Y = y_k|X_1 \dots X_n) = \frac{P(Y = y_k) \prod_i P(X_i|Y = y_k)}{\sum_j P(Y = y_j) \prod_i P(X_i|Y = y_j)} \quad (2)$$

Equation (2) is the fundamental equation for the Naive Bayes classifier. Given a new instance $X^{new} = \langle X_1 \dots X_n \rangle$, this equation shows how to calculate the probability that Y will take on any given value, given the observed attribute values of X^{new} and given the distributions $P(Y)$ and $P(X_i|Y)$ estimated from the training data. If we are interested only in the most probable value of Y , then we have the Naive Bayes classification rule:

$$Y \leftarrow \arg \max_{y_k} \frac{P(Y = y_k) \prod_i P(X_i|Y = y_k)}{\sum_j P(Y = y_j) \prod_i P(X_i|Y = y_j)}$$

which simplifies to the following (because the denominator does not depend on y_k).

$$Y \leftarrow \arg \max_{y_k} P(Y = y_k) \prod_i P(X_i | Y = y_k) \quad (3)$$

2.3 Naive Bayes for Discrete-Valued Inputs

To summarize, let us precisely define the Naive Bayes learning algorithm by describing the parameters that must be estimated, and how we may estimate them.

When the n input attributes X_i each take on J possible discrete values, and Y is a discrete variable taking on K possible values, then our learning task is to estimate two sets of parameters. The first is

$$\theta_{ijk} \equiv P(X_i = x_{ij} | Y = y_k) \quad (4)$$

for each input attribute X_i , each of its possible values x_{ij} , and each of the possible values y_k of Y . Note there will be nJK such parameters, and note also that only $n(J-1)K$ of these are independent, given that they must satisfy $1 = \sum_j \theta_{ijk}$ for each pair of i, k values.

In addition, we must estimate parameters that define the prior probability over Y :

$$\pi_k \equiv P(Y = y_k) \quad (5)$$

Note there are K of these parameters, $(K-1)$ of which are independent.

We can estimate these parameters using either maximum likelihood estimates (based on calculating the relative frequencies of the different events in the data), or using Bayesian MAP estimates (augmenting this observed data with prior distributions over the values of these parameters).

Maximum likelihood estimates for θ_{ijk} given a set of training examples D are given by

$$\hat{\theta}_{ijk} = \hat{P}(X_i = x_{ij} | Y = y_k) = \frac{\#D\{X_i = x_{ij} \wedge Y = y_k\}}{\#D\{Y = y_k\}} \quad (6)$$

where the $\#D\{x\}$ operator returns the number of elements in the set D that satisfy property x .

One danger of this maximum likelihood estimate is that it can sometimes result in $\hat{\theta}$ estimates of zero, if the data does not happen to contain any training examples satisfying the condition in the numerator. To avoid this, it is common to use a “smoothed” estimate which effectively adds in a number of additional “hallucinated” examples, and which assumes these hallucinated examples are spread evenly over the possible values of X_i . This smoothed estimate is given by

$$\hat{\theta}_{ijk} = \hat{P}(X_i = x_{ij} | Y = y_k) = \frac{\#D\{X_i = x_{ij} \wedge Y = y_k\} + l}{\#D\{Y = y_k\} + lJ} \quad (7)$$

where J is the number of distinct values X_i can take on, and l determines the strength of this smoothing (i.e., the number of hallucinated examples is lJ). This expression corresponds to a MAP estimate for θ_{ijk} if we assume a Dirichlet prior

distribution over the θ_{ijk} parameters, with equal-valued parameters. If l is set to 1, this approach is called Laplace smoothing. See the chapter on Estimating Probabilities.

Maximum likelihood estimates for π_k are

$$\hat{\pi}_k = \hat{P}(Y = y_k) = \frac{\#D\{Y = y_k\}}{|D|} \quad (8)$$

where $|D|$ denotes the number of elements in the training set D .

Alternatively, we can obtain a smoothed estimate, or equivalently a MAP estimate based on a Dirichlet prior over the π_k parameters assuming equal priors on each π_k , by using the following expression

$$\hat{\pi}_k = \hat{P}(Y = y_k) = \frac{\#D\{Y = y_k\} + l}{|D| + lK} \quad (9)$$

where K is the number of distinct values Y can take on, and l again determines the strength of the prior assumptions relative to the observed data D .

2.4 Naive Bayes for Continuous Inputs

In the case of continuous inputs X_i , we can of course continue to use equations (2) and (3) as the basis for designing a Naive Bayes classifier. However, when the X_i are continuous we must choose some other way to represent the distributions $P(X_i|Y)$. One common approach is to assume that for each possible discrete value y_k of Y , the distribution of each continuous X_i is Gaussian, and is defined by a mean and standard deviation specific to X_i and y_k . In order to train such a Naive Bayes classifier we must therefore estimate the mean and standard deviation of each of these Gaussians:

$$\mu_{ik} = E[X_i|Y = y_k] \quad (10)$$

$$\sigma_{ik}^2 = E[(X_i - \mu_{ik})^2|Y = y_k] \quad (11)$$

for each attribute X_i and each possible value y_k of Y . Note there are $2nK$ of these parameters, all of which must be estimated independently.

Of course we must also estimate the priors on Y as well

$$\pi_k = P(Y = y_k) \quad (12)$$

The above model summarizes a Gaussian Naive Bayes classifier, which assumes that the data X is generated by a mixture of class-conditional (i.e., dependent on the value of the class variable Y) Gaussians. Furthermore, the Naive Bayes assumption introduces the additional constraint that the attribute values X_i are independent of one another within each of these mixture components. In particular problem settings where we have additional information, we might introduce additional assumptions to further restrict the number of parameters or the complexity of estimating them. For example, if we have reason to believe that noise in the